

PROYECTO INTERMODULAR

ONE PAGE WEB DE INSCRIPCIÓN Y GESTIÓN PARA UNA CARRERA CICLISTA

Alumno: **Francisco José García Franco**

1º Técnico DAM - UAX

Curso: **2025/2026**

Entrega: **04/06/2026**

MEMORIA DE PROYECTO INTERMODULAR 1º DAM PLATAFORMA ONE PAGE WEB DE INSCRIPCIÓN Y GESTIÓN PARA UNA CARRERA CICLISTA

Autor: Francisco José García Franco

Titulación: Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (DAM)

Centro: Universidad Alfonso X el Sabio (UAX)

Curso: 1º (2025-2026)

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y DESCRIPCIÓN BÁSICA DEL PROYECTO

- 1.1. Contexto y encuadre académico
- 1.2. Descripción básica y requisitos del proyecto solicitado
- 1.3. Descripción básica del proyecto presentado y bloques principales
- 1.4. El Efecto *Parallax* y funcionamiento esencial
- 1.5. Funcionalidades Extra fuera de la *One-Page*

2. TECNOLOGÍAS Y ENTORNO DE DESARROLLO EMPLEADOS

- 2.1. Tecnologías del Lado del Cliente (*Frontend*)
- 2.2. Tecnologías del Lado del Servidor y Capa de Datos (*Backend*)
- 2.3. Entorno de Servidor Local y Migración de Infraestructura (XAMPP / Laragon)

3. ESTRUCTURA Y DISEÑO DE LA BASE DE DATOS

- 3.1. Tabla categorías
- 3.2. Tabla participantes
- 3.3. Interacción de las tablas con la aplicación web
- 3.4. Creación y Configuración del Entorno en phpMyAdmin
- 3.5. Gestión de Usuarios y Políticas de Seguridad

4. ARQUITECTURA DE ARCHIVOS Y ORGANIZACIÓN DEL PROYECTO

- 4.1. Árbol de directorios y descripción de los componentes del sistema
- 4.2. Justificación Estética y Selección de la Paleta de Colores
- 4.3. Justificación del Rendimiento de Interfaz mediante *System Font Stack* (Tipografía)

5. EXPLICACIÓN DEL CÓDIGO Y LÓGICA DEL PROYECTO

- 5.1. Conexión e Inicio de Sesión (db.php y control de persistencia con session_start)
- 5.2. Lógica Dinámica de la Portada, consulta asociativa y cálculo de aforo en vivo (index.php)
- 5.3. Biblioteca jQuery: funciones nativas empleadas y control automatizado de luminosidad
- 5.4. Maquetación Eficiente y Diseño Web Adaptativo mediante rejillas fluidas (*Responsive - CSS3*)
- 5.5. Flujo y Procesamiento del Registro, saneamiento y validación (registrar.php)
- 5.6. Control del Panel de Organización, verificación y gestión de bajas (admin.php / eliminar.php)

6. CONCLUSIONES Y LÍNEAS DE MEJORA

- 6.1. Conclusiones del Proyecto
- 6.2. Líneas de Trabajo Futuras (Posibles Mejoras)

1. INTRODUCCIÓN Y DESCRIPCIÓN BÁSICA DEL PROYECTO

1.1 Contexto y encuadre académico.

El presente trabajo se encuadra dentro de la entrega de un proyecto intermodular de 1º de Técnico de Desarrollo de Aplicaciones Multiplataforma cursado por el alumno en FP de la Universidad Alfonso X (UAX).

1.2 Descripción básica y requisitos del proyecto solicitado

- Creación de una plataforma **web One-Page** con secciones bien diferenciadas.
- Elección libre de la temática.
- Uso de los conocimientos adquiridos en **programación**, lenguaje de marcas (**HTML y CSS**), **base de datos**, **sistemas informáticos** y **entornos de desarrollo**.
- Utilización de efecto **parallax** y alguna **animación** empleando **CSS**.
- Empleo de un **entorno de desarrollo tipo XAMPP** que permita emular un servidor real.
- El contenido de la página se cargará directamente de la **base de datos MySQL** con programación **php**.

1.3 Descripción básica del proyecto presentado y bloques principales

- One page web para la gestión integral de inscripciones del **evento ciclista "Gran Fondo Cartagena 2026"**.
- La web cumple los requisitos solicitados.
- El diseño se inspira en otras páginas web de eventos deportivos similares, adaptando el **aspecto visual** a un evento ficticio en recuerdo al antiguo **equipo ciclista Reynolds de la década de los 80**, tomando los colores corporativos como inspiración.

Toda la información pública para el usuario se concentra en una única página (index.php), dividida en estos **bloques principales**:

- **Barra de Navegación Superior:** fija en la parte alta de la pantalla. Con el nombre del evento, un **menú** desplegable para saltar directamente a cualquiera de las **secciones** (Inicio, Recorridos, Servicios e Inscripciones) sin tener que hacer *scroll* manual, y un **enlace** para cambiar el diseño entre **Modo Claro** y **Modo Oscuro**.
- **Cabecera / Portada:** contiene el título del evento ("Gran Fondo Cartagena"), la fecha y un botón para ir directo a la sección Inscripciones.
- **La Experiencia (Métricas y Recorridos):** muestra los datos clave de la carrera (**distancias, desniveles**) y unas **tarjetas interactivas** que giran al pasar el ratón para mostrar los detalles y precios de las tres rutas disponibles. Incluye un marcador que indica el número de personas inscritas.
- **Servicios:** explicación visual de servicios incluidos (avituallamientos, maillot de regalo, comida) y las recomendaciones de hoteles asociados.
- **Boletín de Inscripción:** **formulario** donde el corredor introduce sus datos, elige su ruta y confirma si necesita o no alojamiento.

1.4 El Efecto Parallax y su funcionamiento esencial

El efecto Parallax se aplica en los **fondos de las secciones principales** (en la portada, en la zona de recorridos, en la sección de servicios y en el formulario).

Funcionamiento esencial: al hacer scroll, las imágenes de fondo se mueven a una velocidad mucho más lenta que los textos y los botones que van por encima. Esto genera una ilusión óptica de profundidad, como si el contenido flotara sobre los fondos mientras se navega en la web.

1.5 Funcionalidades Extra fuera de la One-Page

Se han añadido funcionalidades fuera de la One-Page, con dos procesos clave en archivos separados.

- **El procesamiento del formulario:** al confirmar la inscripción, esta es manejada en otro archivo **php** que recoge los datos, comprueba que todo esté correcto, los guarda en la base de datos y devuelve al usuario a la página principal mostrando un mensaje.
- **El panel de consulta del organizador:** una pantalla totalmente oculta para el usuario normal. En el pie de página hay un enlace discreto que, a través de una contraseña lleva al organizador a una pantalla nueva con un panel que le permite ver la lista de inscritos y borrar los que desee, permitiendo a través de un botón volver a la página inicial.

2. TECNOLOGÍAS Y ENTORNO DE DESARROLLO EMPLEADOS

Conjunto de tecnologías empleadas en el desarrollo del proyecto:

2.1. Tecnologías del Lado del Cliente (Frontend)

- **HTML5:** para construir el esqueleto de la página web, estructurando de manera ordenada el menú, los textos, los bloques de información y las casillas del formulario de inscripción.
- **CSS3:** para implementar la capa visual y el diseño estético de la web, permitiendo aplicar los colores corporativos inspirados en el equipo Reynolds, gestionar el tamaño y orden de los elementos, crear el efecto Parallax de los fondos y programar el giro en tres dimensiones de las tarjetas de los recorridos.
- **JavaScript y jQuery:** empleadas para dar dinamismo a la web permitiendo alternar entre el Modo Claro y el Modo Oscuro, además de activar este último automáticamente leyendo la hora del sistema.

2.2. Tecnologías del Lado del Servidor y Datos (Backend)

- **PHP:** como lenguaje encargado de la lógica interna del servidor. Se emplea para leer dinámicamente los recorridos guardados en la base de datos, calcular las plazas disponibles, validar la validez de datos introducidos en el formulario y redirigir al usuario mostrando las alertas de éxito o error en el proceso de inscripción.
- **MySQL:** El motor relacional escogido para almacenar de manera estructurada y segura toda la información del evento, dividida en las tablas de rutas y participantes.

2.3. Entorno de Servidor Local (XAMPP / Laragon)

Dado que los archivos PHP y las bases de datos MySQL no pueden ejecutarse abriéndolos directamente en el navegador de forma normal, se ha utilizado un **entorno de desarrollo local que admiten lenguaje php**. Se comenzó el proyecto empleando **XAMPP**, pero después de problemas varios que dieron lugar al bloqueo total e imposibilidad de uso se pasó el desarrollo a la aplicación **Laragon**.

La aplicación funciona instalando de manera local en el ordenador un **servidor web Apache** y el módulo de **MySQL**. De este modo, emula a la perfección el comportamiento de un servidor en internet real, permitiendo testear las conexiones del archivo **db.php** que contiene los parámetros de conexión y procesar los envíos del formulario entre otros.

3. ESTRUCTURA Y DISEÑO DE LA BASE DE DATOS

Para dar soporte a la carga dinámica de la web y registrar a los usuarios, el sistema se conecta a una **base de datos relacional creada en MySQL llamada granfondo cartagena**. Se trata de una base de datos sencilla con dos tablas principales conectadas entre sí mediante el empleo de claves foráneas:

3.1. Tabla categorías

Tabla **estática** del sistema que **funciona a modo de catálogo**, guardando la información de los tres recorridos que ofrece el evento. Posibles futuros cambios por parte de la organización en el precio o la distancia de las rutas se gestionarían modificando esta tabla, actualizándose automáticamente en toda la web.

Consta de los siguientes campos:

- **id** (INT AUTO_INCREMENT): tipo de dato entero (INT), clave primaria. Se le añade la propiedad AUTO_INCREMENT para que el propio motor de MySQL genere un número correlativo automáticamente con cada registro.
- **nombre** (VARCHAR(100)): cadena de texto de longitud variable (VARCHAR) limitada a 100 caracteres. Es la opción ideal para almacenar los nombres de los circuitos, ya que consume únicamente el espacio del texto real introducido, dejando un margen suficiente por si los nombres de los homenajes ciclistas son largos.
- **distancia** (VARCHAR(50)): cadena de texto de longitud variable (VARCHAR) de hasta 50 caracteres porque, además del valor numérico, se guarda la unidad de medida "Km" (ej. "180 Km"). Al no requerir operaciones matemáticas, el formato texto facilita su lectura directa y renderizado en la web.
- **desnivel** (VARCHAR(50)): cadena de texto de longitud variable VARCHAR de 50 caracteres. Su elección se debe a que almacena el número junto con la unidad de metros (ej. "3500 m"), sirviendo puramente como un dato informativo para el usuario.
- **precio** (DECIMAL(6,2)): se descartan los tipos flotantes comunes por sus problemas de redondeo y se selecciona DECIMAL(6,2). Este tipo garantiza una precisión matemática exacta para valores monetarios. El parámetro 6,2 significa que el campo admite un total de 6 dígitos, de los cuales 2 están reservados estrictamente para los céntimos (permitiendo importes de hasta 9999,99 €).

3.2. Tabla participantes

Tabla **dinámica** del proyecto, donde se almacena la **información personal introducida por cada usuario** que completa el boletín de inscripción.

Los campos que la componen son:

- **id** (INT AUTO_INCREMENT): tipo de dato entero (INT), clave primaria. Se le añade la propiedad AUTO_INCREMENT para que el propio motor de MySQL genere un número correlativo automáticamente con cada registro.
- **nombre** (VARCHAR(100)) y **apellidos** (VARCHAR(100)): cadenas de texto de longitud variable VARCHAR con un límite de 100 caracteres cada uno. Esta longitud es el estándar óptimo en bases de datos para garantizar que quepan nombres y apellidos compuestos sin riesgo de truncar la información en el servidor.
- **dni** (VARCHAR(20) NOT NULL UNIQUE): cadena de texto de longitud variable VARCHAR de hasta 20 espacios. Se define como UNIQUE (único) al ser una regla de negocio crítica: dos corredores no pueden inscribirse con el mismo documento de identidad, evitando duplicados accidentales en la lista de la carrera.
- **email** (VARCHAR(150)): cadena de texto de longitud variable VARCHAR con un límite amplio de 150 caracteres. Esto asegura el soporte para correos electrónicos corporativos o personales inusualmente largos.
- **id_categoria** (INT): número entero (INT) para almacenar el número de ID de la ruta elegida, permitiendo declararlo como clave foránea (Foreign Key) para establecer la relación con la tabla categorías.
- **alojamiento** (VARCHAR(2) DEFAULT 'No'): cadena de texto de longitud variable VARCHAR muy corto, limitado a solo 2 caracteres. Guardará únicamente las respuestas "Si" o "No", consumiendo el mínimo espacio posible en el disco. Además, incluye la propiedad DEFAULT 'No' para indicar la opción de solo carrera (sin alojamiento) por defecto si no se marca otra cosa.

- **fecha_registro** (TIMESTAMP DEFAULT CURRENT_TIMESTAMP): tipo de dato específico para registrar instantes exactos de tiempo. Con la propiedad DEFAULT CURRENT_TIMESTAMP se toma como valor en el instante preciso en que la inscripción entra en la base de datos.

3.3. Interacción de las tablas con la aplicación web

Las dos tablas interactúan con los archivos de la plataforma dependiendo de la acción realizada:

- **Carga de datos en la Portada** (index.php): al abrir la web, el código PHP realiza una consulta para leer la tabla categorías y pintar las tres tarjetas visuales y las opciones del formulario. Además, realiza un recuento del número de filas de la tabla participantes para mostrar en vivo el número de inscritos y de plazas libres.
- **Inserción de datos (registrar.php)**: al enviar el formulario, este archivo vuelca los datos introducidos directamente en una nueva fila dentro de la tabla participantes.
- **Control y borrado (admin.php / eliminar.php)**: en el acceso al panel privado de la organización, el sistema lee la tabla participantes para mostrarle la lista completa en pantalla. Si el organizador decide pulsar el botón de borrar, el archivo eliminar.php quita ese registro específico de la tabla de forma permanente.

3.4. Creación y Configuración del Entorno en phpMyAdmin

Para el montaje físico de esta estructura de datos en el servidor local administrado por Laragon, se accedió al panel de control web de phpMyAdmin y desde la pestaña "SQL", se ejecutó el siguiente código para automatizar la construcción de las tablas, la carga inicial de información y las políticas de acceso:

```
-- 1. CREACIÓN DE LA BASE DE DATOS
CREATE DATABASE granfondo_cartagena;
USE granfondo_cartagena;

-- 2. TABLA DE CATEGORÍAS (Para la carga dinámica de rutas y precios)
CREATE TABLE categorias (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  distancia VARCHAR(50) NOT NULL,
  desnivel VARCHAR(50) NOT NULL,
  precio DECIMAL(6,2) NOT NULL
);

-- 3. TABLA DE PARTICIPANTES (Gestión y almacenamiento de inscripciones)
CREATE TABLE participantes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  dni VARCHAR(20) NOT NULL UNIQUE,
  email VARCHAR(150) NOT NULL,
  id_categoria INT NOT NULL,
  alojamiento VARCHAR(2) NOT NULL DEFAULT 'No',
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_categoria) REFERENCES categorias(id)
);

-- 4. INSERCIÓN DE DATOS INICIALES (Circuitos inspirados en Mallorca 312 y Reynolds)
INSERT INTO categorias (nombre, distancia, desnivel, precio) VALUES
('Gran Fondo Perico Delgado', '180 Km', '3500 m', 85.00),
('Medio Fondo Miguel Induráin', '120 Km', '2100 m', 65.00),
('Mini Fondo Puertos de Mazarrón', '75 Km', '1100 m', 45.00);

-- 5. PROTOCOLO DE USUARIOS DEL SISTEMA, SEGURIDAD Y PRIVILEGIOS

-- Crear Usuario de conexión para la Aplicación Web (Permisos de inserción, lectura y borrado)
CREATE USER 'cliente_granfondo_ct'@'localhost' IDENTIFIED BY 'cl5ent4.!Reynolds1980';
-- Asignar los permisos específicos (Lectura, Inserción y Borrado) sobre la Base de Datos
GRANT SELECT, INSERT, DELETE ON granfondo_cartagena.* TO 'cliente_granfondo_ct'@'localhost';
-- Aplicar los cambios en el sistema de permisos
FLUSH PRIVILEGES;
```

3.5. Gestión de Usuarios y Políticas de seguridad

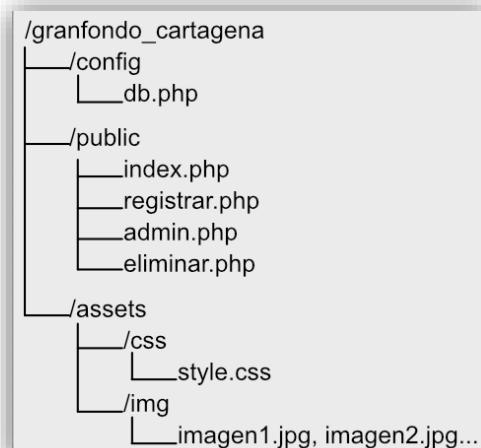
Como sabemos, los entornos locales como XAMPP o Laragon tienen configurado de fábrica un usuario administrador universal llamado **root sin contraseña**. Su uso en la versión final de un software es una grave falta de seguridad.

Para subsanar este riesgo y **blindar el acceso a la base de datos**, se generó un **usuario dedicado para esta aplicación**, con los siguientes principios:

- **Nomenclatura no predecible:** En lugar de emplear nombres genéricos como admin o root, se crea el usuario '**cliente_granfondo_ct**', lo que dificulta los ataques automatizados de fuerza bruta para adivinar el acceso al servidor.
- **Uso de Contraseña Robusta:** Se ha protegido la cuenta con la clave '**cl5ent4.!Reynolds1980**' que combina letras, números y caracteres especiales, garantizando que nadie pueda conectar con la base de datos sin una autenticación estricta.
- **Privilegios Acotados de Operación:** La instrucción **GRANT limita las acciones de este usuario** a lo estrictamente necesario para el funcionamiento de la web (seleccionar datos para pintarlos en tarjetas o tablas, insertar inscripciones del formulario y borrar filas desde el panel del organizador). Al no otorgarle permisos totales (como UPDATE o DROP), si un atacante lograra vulnerar el formulario, el propio motor de MySQL le impediría destruir la base de datos o modificar los datos.

4. ARQUITECTURA DE ARCHIVOS Y ORGANIZACIÓN DEL PROYECTO

El proyecto se ha estructurado conforme al siguiente árbol de directorios, separando la configuración privada del servidor de los archivos públicos que maneja el navegador web:



4.1. Árbol de directorios y descripción de los componentes del sistema

La función asignada a cada archivo y su papel dentro del funcionamiento global de la plataforma son los siguientes:

- **db.php (Conexión a la base de datos):** archivo que se encarga de abrir la comunicación con MySQL mediante la abstracción **PDO**. Se ubica en una carpeta separada e independiente para proteger sus credenciales y configuraciones frente a accesos ajenos.
- **index.php (Interfaz del Usuario):** es el núcleo visual del proyecto. Carga desde la base de datos los recorridos, realiza los cálculos de aforo y captura los datos a través del formulario de registro. Integra:
 - todos los elementos **HTML** de la one web page.
 - el código de **jQuery** para gestionar los Modos Claro y Oscuro.
- **registrar.php (Gestor de registros):** carece de parte visual. Es un script que se ejecuta únicamente cuando se envía un formulario. Lee los datos remitidos mediante el protocolo **POST**, comprueba que el formato del email sea el correcto, limpia los textos, inserta la información en la tabla participantes y devuelve al corredor a la portada inyectando un mensaje de éxito en la sesión.
- **admin.php (Gestión de inscritos por la organización):** página exclusiva para el organizador de la carrera a la que se ingresa mediante clave. A través del usuario de la base de datos creado para la web y mediante **db.php** se conecta a dicha base de datos y realiza una consulta, montando una tabla estructurada para mostrar los usuarios registrados con sus campos. Los estilos son propios e independientes, habiéndose incrustado en su cabecera mediante etiqueta `<style>` para separarlo de la hoja de estilos asociada al index.php.
- **eliminar.php (Gestor de Bajas por la organización):** es un complemento al panel de administración. Cuando se clicca en el enlace "Eliminar" sobre un corredor, este archivo recoge el ID de la fila a través de la URL, ejecuta la orden de **borrado en MySQL** y refresca la pantalla instantáneamente devolviendo al gestor al panel actualizado.
- **style.css (Hoja de Estilos):** centraliza la **maquetación**, empleándose un **diseño web responsive**. Define una serie de variables para almacenar los colores empleados en la web, permitiendo su reutilización a lo largo del código. Además, recoge las reglas para las transiciones de color de los modos claro/oscuro, los tiempos del efecto **Parallax** de fondo y el comportamiento en tres dimensiones de los giros de las **tarjetas**.

4.2. Justificación Estética y Selección de la Paleta de Colores

Para dar un carácter coherente a la estética de todo el proyecto conforme a los **colores del Equipo Reynolds** se obtuvieron los colores de la fotografía de un maillot de la década de los 80, arrojando los códigos hexadecimales **#0b2545** para el Azul Marino Reynolds y **#d62828** para el Rojo de Competición.

Se completó la paleta para dar armonía al conjunto, seleccionando un azul secundario más eléctrico (**#134074**) para hacer de puente visual. Se eligió un color cálido análogo al rojo: Crema/Dorado clásico (**#f4a261**), idóneo para destacar fechas y métricas.

Finalmente se añadieron los fondos gris claro (**#eef4f8**) y oscuro puro (**#141414**) para garantizar que, al conmutar entre el Modo Claro y el Modo Oscuro, los elementos de la interfaz mostraran un contraste adecuado.

4.3. Justificación del Rendimiento de Interfaz mediante System Font Stack (Tipografía)

Se seleccionó la **familia tipográfica** atendiendo a un criterio puramente técnico, enfocado en la optimización del **rendimiento** y la **velocidad de carga** de la página web.

En lugar de tipografías pesadas desde servidores externos (como Google Fonts), lo cual ralentizaría la carga inicial en el servidor local (Laragon) y penalizaría las conexiones móviles, se configuró en CSS una estrategia llamada **System Font Stack** (pila de fuentes del sistema), mediante el código:

font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;

Esta instrucción hace que el navegador no descargue nada, sino que emplee la fuente de máxima calidad que el usuario tiene instalada de fábrica en su sistema operativo:

- En **Apple** (Mac/iPhone), la web se renderiza con la fuente **San Francisco** (-apple-system).
- En **Windows**, utiliza la tipografía oficial de Microsoft (**'Segoe UI'**).
- En **Android**, llama a la fuente nativa de Google (**Roboto**).

Como seguridad final (fallback), si el sistema no encuentra ninguna, recurre a **Arial** o **Helvetica**, garantizando un diseño limpio, plano (sans-serif) y con una legibilidad impecable en el 100% de las pantallas.

5. EXPLICACIÓN DEL CÓDIGO Y LÓGICA DEL PROYECTO

Breve descripción de la programación que define el comportamiento interno de la aplicación, analizando el flujo de la información en las interacciones usuario-servidor.

5.1. Conexión e Inicio de Sesión (db.php y control de persistencia con session_start)

Para que cualquier archivo del sistema pueda hablar con la base de datos, lo primero que hace es incluir el archivo **db.php** usando la instrucción **require_once**.

Dentro de **db.php** se crea un **objeto de conexión** llamado **PDO**, estándar moderno en PHP que incluye un control estricto de errores: si la base de datos se cae o el usuario de seguridad no se identifica correctamente, el sistema frena la ejecución de golpe y lanza una excepción controlada para evitar que la web muestre código interno comprometido en la pantalla del navegador.

Por otro lado, tanto en la portada (**index.php**) como en el procesador (**registrar.php**) se arranca el motor de sesiones del servidor con **session_start()**, lo que permite habilitar los "mensajes flash": **textos guardados temporalmente** en la memoria del servidor para recordar entre pantallas/archivos la información acerca del éxito o error en las inscripciones.

5.2. Lógica Dinámica de la Portada, consulta asociativa y cálculo de aforo en vivo (index.php)

Cuando la página se carga en el navegador, ocurren dos procesos antes de mostrar el contenido:

- **Pintado de recorridos:** php lanza una consulta a la tabla **categorias**, guarda las filas encontradas en una variable **\$categorias** y, mediante un bucle **foreach** incrustado en el HTML, fabrica las tres tarjetas interactivas, rellenando además las opciones del menú desplegable del formulario.
- **Cálculo de plazas en vivo:** en el archivo se cuentan las filas totales de la tabla participantes con la función **COUNT(*) de SQL**. El número se guarda en la variable **\$totalInscritos**. Después, restando ese valor al aforo máximo fijado por la organización (500 plazas, constante definida en cabecera del archivo), se obtiene la variable **\$plazasDisponibles**. Ambas cifras se inyectan directamente en los contadores del boletín de inscripción.

5.3. Biblioteca jQuery: funciones nativas empleadas y control automatizado de luminosidad

Se emplea la biblioteca **jQuery** para simplificar la sintaxis de **JavaScript** y controlar **interacciones en tiempo real** dentro del navegador, utilizando las siguientes funciones nativas:

- **\$(document).ready(function() { ... })**: asegura que ningún script de la página se ejecute antes de tiempo. Espera a que todo el esqueleto HTML esté completamente descargado y listo, garantizando que los botones y los menús respondan al instante cuando el usuario interactúe con ellos.
- **\$('#toggleMode').click(function(e) { ... })**: modifica el comportamiento por defecto del enlace de cambio de aspecto. Al detectar el clic del usuario, ejecuta la función anónima, frenando la recarga de la página con **e.preventDefault()** y ejecuta la lógica para comprobar qué clase visual tiene la web en ese momento y cambiarla a la contraria.
- **hasClass(), removeClass() y addClass()**: funciones para realizar el cambio de modos de luminosidad. **hasClass('modo-claro')** comprueba si la web está en modo diurno. Dependiendo del resultado, las otras dos funciones retiran una clase del body e insertan la otra en milisegundos, permitiendo que la hoja de estilos cambie todos los colores de la pantalla a la vez.
- **text()**: para modificar de manera dinámica el texto visible del botón de la barra superior, alternando entre los strings "Modo Claro" o "Modo Oscuro" para informar adecuadamente al usuario de la acción que realizará en su siguiente clic.

El cambio entre el **Modo Claro** y el **Modo Oscuro** se controla mediante funciones de jQuery. Al arrancar, el script ejecuta una lógica de control doble:

- **Control manual:** escucha, mediante el método `$('#toggleMode').click(function(e) { })` si el usuario hace clic en el botón "Modo Oscuro" de la barra superior. Si lo pulsa, jQuery detecta si el cuerpo de la web (body) tiene la clase `.modo-claro`. En ese mismo instante, retira esa clase, añade `.modo-oscuro` y cambia el texto del botón a "Modo Claro".
- **Control automático por horario:** mediante la función `getHours()` de **JavaScript**, la web lee la hora interna del reloj del ordenador del usuario. Si la hora es igual o mayor a las **20:00** de la tarde, o menor a las **08:00** de la mañana, la web arranca directamente aplicando el Modo Oscuro sin que el usuario tenga que pulsar nada. Durante el resto de horas arranca en Modo Claro.

5.4. Maquetación Eficiente y Diseño Web Adaptativo mediante rejillas fluidas (Responsive - CSS3)

Se ha diseñado una estructura fluida basada en **Flexbox** para asegurar que la interfaz visual de la web se adapte y sea legible en pantallas de ordenador, tablets y teléfonos móviles.

Se emplean **rejillas flexibles** (rejilla-flex) combinadas con la función `calc()` de **CSS**. Por ejemplo, en pantallas grandes las tarjetas de los recorridos se muestran en filas de tres columnas, ocupando cada una un tercio del espacio disponible (`calc(33.333% - 20px)`). Gracias a la propiedad `flex-wrap: wrap` y a un ancho mínimo fijado por seguridad (`min-width: 260px`), cuando la pantalla se estrecha las columnas que dejan de caber a lo ancho se reordenan en una columna.

Para los textos principales de la portada se ha utilizado la función tipográfica moderna **font-size: clamp()**, que permite definir un tamaño de letra dinámico que se calcula en tiempo real al el ancho del navegador.

5.5. Flujo y Procesamiento del Registro, saneamiento y validación (registrar.php)

Cuando se pulsa "**Confirmar Inscripción**", los datos viajan hacia **registrar.php** en un paquete cerrado (método **POST**). El archivo realiza estas tareas críticas en orden:

- **Limpieza y Saneamiento:** pasa todos los textos por la función `trim()` de php, borrando los espacios en blanco que el usuario haya podido introducir sin pretensión al principio o al final de las casillas, evitando guardar datos sucios en la base de datos.
- **Validación:** aplica filtros internos, comprobando que el correo tenga una estructura real (@ y un dominio válido) mediante la función nativa de **php** `filter_var($email, FILTER_VALIDATE_EMAIL)`. Si alguna casilla obligatoria llega vacía o el email es inventado, el script interrumpe el proceso, guarda un texto de queja en `$_SESSION['msg_error']` y devuelve al usuario a la portada.
- **Inserción Segura:** si las validaciones son correctas se prepara la orden **INSERT INTO** usando **prepared statemets**, evitando introducir los textos del usuario directamente en la frase de SQL, mediante marcadores intermedios. Esto hace que MySQL trate la información como texto puro, impidiendo la inyección de código SQL para hackear el sistema. Tras guardar al usuario se genera el texto de éxito en `$_SESSION['msg_exito']`.
- **Redirección:** el script termina con la instrucción `header("Location: index.php#inscripciones")`. Esto saca al usuario del archivo y lo devuelve a la `index.php`, obligando al navegador a bajar la pantalla directamente hasta el formulario gracias al ancla `#inscripciones`. De este modo se direcciona al usuario para que vea la alerta verde o roja de su estado sin tener que buscarla haciendo scroll.

5.6. Control del Panel de Organización, verificación y gestión de bajas (admin.php / eliminar.php)

- **Filtro por Contraseña Visual:** en el pie de página, el enlace al panel tiene un disparador **onclick** con una pequeña lógica en **JavaScript**. Al pulsar, salta una ventana emergente **prompt()** pidiendo la clave de la organización, que es '**Reynolds1980**'; en caso de introducción correcta, la función devuelve true y el navegador abre admin.php. Si la clave falla, devuelve false, lo que bloquea el enlace por completo e impide que el navegador intente siquiera cargar el archivo privado.
- **Construcción de la tabla:** ya dentro de **admin.php**, el código realiza una consulta de lectura uniendo los datos de las tablas. Mediante un bucle se genera dinámicamente una fila de tabla HTML por cada registro, mostrando ordenadamente sus apellidos, nombre, DNI y opciones. Cada fila incluye un botón rojo de "Eliminar" vinculado al ID de ese registro.
- **Lógica de borrado (eliminar.php):** si se pulsa "Eliminar", se viaja a eliminar.php enviando el número de ID a través de la barra de direcciones (método GET). El script recoge ese número, lanza una orden de borrado (**DELETE FROM participantes WHERE id = :id**) mediante **prepared statement** de **PDO** y redirige de vuelta a admin.php, haciendo que la tabla se refresque en pantalla mostrando la lista real sin el participante eliminado.

6. CONCLUSIONES Y LÍNEAS DE MEJORA

Se presenta un balance global sobre el desarrollo de la plataforma web y se plantean posibles mejoras futuras para la misma.

6.1. Conclusiones del Proyecto

La realización de este proyecto intermodular ha resultado clave para afianzar de manera práctica y conjunta las competencias adquiridas durante el primer curso del ciclo formativo de Grado Superior en DAM:

- De modo global, mediante el proyecto se ha conseguido una **integración** de conocimientos de los ámbitos de las distintas asignaturas de primer curso de DAM: maquetación web responsive (**Lenguajes de Marcas**) con persistencia de datos relacionales en MySQL (**Bases de Datos**) y control lógico de flujos de información en **php** y **JavaScript**, con un funcionamiento de arquitectura **Cliente-Servidor**.
- Durante el desarrollo surgieron situaciones habituales del entorno laboral real, como la **gestión de servidores locales**, el bloqueo de puertos en **XAMPP** que obligó a migrar con éxito a **Laragon**, y la optimización de código para evitar duplicidades innecesarias en las hojas de estilo.
- Se ha adoptado una **visión** de programación pensando en el **rendimiento** y en la **seguridad informática**, aplicando principios de protección mediante **prepared statement** contra ataques de **inyección de SQL** y políticas de **restricción de usuarios en bases de datos**.

6.2. Líneas de Trabajo Futuras (Posibles Mejoras)

La plataforma es un diseño básico de una aplicación de negocio que admite mejoras en campos diversos, de cara a un entorno comercial y de producción real.

Algunas mejoras posibles serían:

- Adición de otras **secciones**: galería de fotos, clasificaciones ...
- **Pasarela de Pago Integrada**: conectar el formulario mediante servicios API con pasarelas de pago seguro online (como Stripe o PayPal) para tramitar el cobro automatizado de la inscripción.
- **Generación Automatizada de Dorsales**: incorporar algún tipo de biblioteca, como FPDF, en el servidor para que el sistema autogenera un archivo PDF con el resguardo oficial y el número de dorsal asignado.
- **Cifrado de Contraseñas y Roles**: sustituir el filtro de contraseña visual de la organización por un sistema de Login robusto en el backend.